

Material complementar

Envio UART do valor das chaves

Conversão BCD, FSM e comunicação serial

Exemplo comentado em VHDL para a DE10-Lite

Prof. Felipe Walter Dafico Pfrimer

Objetivo

Este documento explica, em detalhes, o exemplo `uart_sw.vhd`: leitura das chaves, conversão binário-BCD, exibição em displays, transmissão serial UART e controle por máquina de estados.

28 de maio de 2026

Sumário

1	Envio UART do valor das chaves	1
1.1	Visão geral	1
1.2	Interface da entidade e parâmetros	2
1.3	Conversão binário-BCD com Double Dabble	3
1.4	Displays de sete segmentos	4
1.5	Transmissor UART 8N1	5
1.6	Geração do baud rate	6
1.7	Máquina de estados de envio	6
1.8	Buffer ASCII e fim de linha	8
1.9	Testbench do exemplo	9
1.10	Relação entre processos e sinais	10
1.11	Pontos de atenção para estudo	11

Capítulo 1

Envio UART do valor das chaves

1.1 Visão geral

O exemplo `uart_sw.vhd` integra quatro ideias importantes em um único circuito: leitura de chaves, conversão numérica, exibição local e comunicação serial. A comunicação serial usa UART, sigla de *Universal Asynchronous Receiver/Transmitter*, isto é, transmissor/receptor assíncrono universal. A entrada principal é o vetor `SW(7 downto 0)`, interpretado como um número binário sem sinal entre 0 e 255. Esse valor é mostrado nos displays de sete segmentos e também transmitido pela UART como texto decimal.

As chaves `SW(9 downto 8)` não fazem parte do valor numérico. Elas selecionam o terminador da linha enviada pela UART: `LF` (*Line Feed*, avanço de linha), `CR` (*Carriage Return*, retorno de carro), `CRLF` (*Carriage Return* seguido de *Line Feed*) ou nenhum terminador. Essa escolha facilita testar o circuito com terminais seriais que esperam convenções diferentes de fim de linha.

Ideia central

O circuito envia uma fotografia periódica das chaves pela UART. Os displays acompanham as chaves continuamente, enquanto a UART captura o valor somente no instante em que prepara uma nova transmissão.

A Figura 1.1 representa o diagrama funcional do exemplo. O mesmo valor lido das chaves alimenta a conversão BCD (*Binary-Coded Decimal*, decimal codificado em binário) por meio do algoritmo Double Dabble. A partir da conversão, um ramo atualiza os displays e outro prepara os caracteres que serão enviados pela UART. Na mesma figura, existem os seguintes termos chaves:

- **FSM** significa *Finite State Machine*, ou máquina de estados finitos;
- **ASCII** significa *American Standard Code for Information Interchange*, o padrão usado para representar caracteres;
- **8N1** indica uma UART com 8 bits de dados, sem paridade e 1 bit de parada.

Espera-se que o leitor já tenha familiaridade com esses conceitos, mas o exemplo é comentado de forma a explicar cada um deles. O código completo do exemplo está disponível no repositório da disciplina, e os trechos mais relevantes são reproduzidos ao longo deste documento.

Caso o leitor queira testar o exemplo, ele pode ser sintetizado e programado na placa DE10-Lite ou adaptado para outras plataformas. Para receber a transmissão UART, pode-se usar um conversor USB-serial e um terminal como PuTTY, Tera Term ou minicom. A

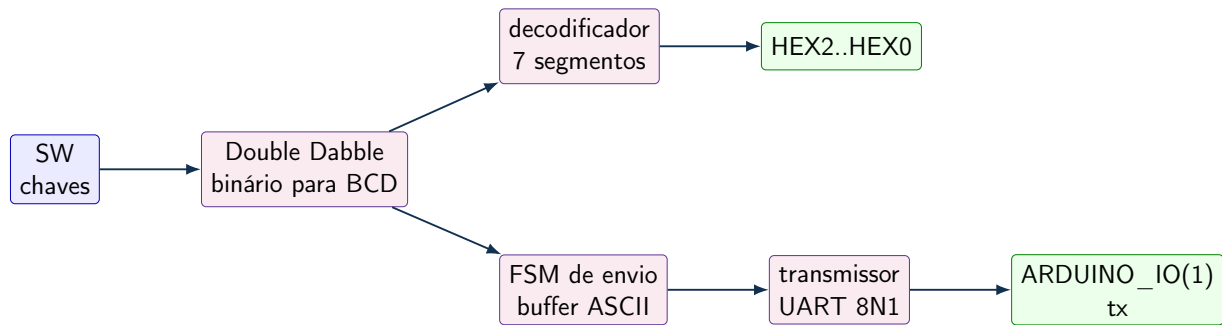


Figura 1.1: Fluxo funcional simplificado do exemplo.

taxa de baud deve ser configurada para 115200, e o terminal deve estar preparado para interpretar os caracteres ASCII enviados.

1.2 Interface da entidade e parâmetros

A entidade foi escrita para ser usada diretamente no kit DE10-Lite, mas pode ser adaptada para outras plataformas. A configuração de pinagem do FPGA pode ser importada para o projeto por meio do arquivo `de10_lite.qsf`, presente na pasta deste exemplo. O clock principal é recebido por `MAX10_CLK1_50`; o botão `KEY(0)` é usado como reset ativo em nível baixo; as chaves `SW` definem o valor transmitido e a opção de fim de linha; a saída serial aparece em `ARDUINO_IO(1)`. Na nomenclatura da placa, `HEX` identifica os displays de sete segmentos e `LEDR` identifica os LEDs vermelhos.

Código 1.1: Interface principal do exemplo `uart_sw`.

```

1 entity uart_sw is
2   generic (
3     frequency : integer := 50000000;
4     baudrate  : integer := 115200;
5     delay_ms  : integer := 200
6   );
7   port (
8     MAX10_CLK1_50 : in   std_logic;
9     KEY           : in   std_logic_vector(1 downto 0);
10    SW             : in   std_logic_vector(9 downto 0);
11    LEDR           : out  std_logic_vector(9 downto 0);
12    ARDUINO_IO     : inout std_logic_vector(15 downto 0);
13    HEX0           : out  std_logic_vector(6 downto 0);
14    HEX1           : out  std_logic_vector(6 downto 0);
15    HEX2           : out  std_logic_vector(6 downto 0);
16  );
17 end entity;

```

Os três *generics* deixam o exemplo ajustável. O parâmetro `frequency` define a frequência do clock de entrada; `baudrate` define a taxa de símbolos da UART; e `delay_ms` define o intervalo entre linhas transmitidas. No uso real com a DE10-Lite, os valores padrão representam clock de 50 MHz, UART a 115200 baud e envio a cada 200 ms.

Sobre baud rate

Em UART, o período de um bit é o inverso da taxa em baud. Para 115200 baud, cada bit dura aproximadamente 8,68 μ s, pois $T_{bit} = 1/115200$ s [1].

1.3 Conversão binário-BCD com Double Dabble

O valor das chaves é naturalmente binário, mas humanos leem com mais facilidade a representação decimal. Para transformar um número binário em dígitos decimais sem usar divisores por 10 no hardware, o exemplo usa o algoritmo Double Dabble, também conhecido como *shift-and-add-3*. A ideia é percorrer os bits do número binário e construir, por deslocamentos, os dígitos BCD correspondentes [3].

No circuito, a função `to_bcd` recebe um vetor de 8 bits e devolve 12 bits. Esses 12 bits são organizados em três grupos de 4 bits:

bcd(11 downto 8) = centena,
bcd(7 downto 4) = dezena,
bcd(3 downto 0) = unidade.

A Figura 1.2 mostra a decisão repetida a cada bit de entrada. Antes do deslocamento, cada dígito BCD é testado; se ele for maior que 4, recebe a soma de 3. Depois disso, o conjunto é deslocado e recebe o próximo bit do número binário.

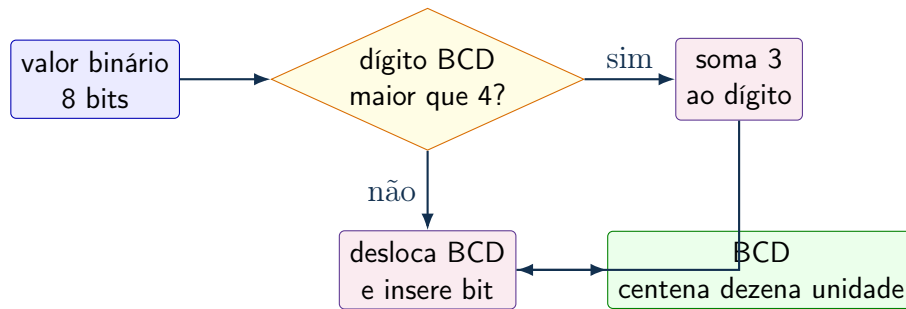


Figura 1.2: Etapa conceitual do Double Dabble para cada bit processado.

Código 1.2: Função de conversão binário-BCD.

```

1 function to_bcd(value : std_logic_vector(7 downto 0)) return std_logic_vector
  is
2   variable bcd_value : unsigned(11 downto 0) := (others => '0');
3 begin
4   for i in 7 downto 0 loop
5     if bcd_value(3 downto 0) > to_unsigned(4, 4) then
6       bcd_value(3 downto 0) := bcd_value(3 downto 0) + to_unsigned(3,
7       4);
8     end if;
9     if bcd_value(7 downto 4) > to_unsigned(4, 4) then

```

```

10         bcd_value(7 downto 4) := bcd_value(7 downto 4) + to_unsigned(3,
11         4);
11     end if;
12
13     if bcd_value(11 downto 8) > to_unsigned(4, 4) then
14         bcd_value(11 downto 8) := bcd_value(11 downto 8) + to_unsigned(3,
15         4);
16     end if;
17     bcd_value := bcd_value(10 downto 0) & value(i);
18 end loop;
19
20 return std_logic_vector(bcd_value);
21 end function;

```

O teste usa a comparação com 4, em vez de escrever “maior ou igual a 5”. Como os dígitos BCD são inteiros de 0 a 9, as expressões > 4 e ≥ 5 são equivalentes. Após a soma de 3, o grupo de 4 bits pode ficar temporariamente maior que 9; isso não é erro. Esse valor intermediário é corrigido pelo deslocamento seguinte, que propaga o ajuste para o próximo dígito decimal.

1.4 Displays de sete segmentos

Além de transmitir o valor pela UART, o exemplo mostra o número nos displays HEX2, HEX1 e HEX0. Essa saída é combinacional: sempre que `data` muda, a função `to_bcd` é chamada em uma variável local e cada dígito BCD é convertido para o padrão de segmentos correspondente.

Código 1.3: *Processo combinacional que atualiza os displays.*

```

1 process(data) is
2     variable display_bcd : std_logic_vector(11 downto 0);
3 begin
4     display_bcd := to_bcd(data);
5
6     HEX0 <= bcd_to_7seg(display_bcd(3 downto 0));
7     HEX1 <= bcd_to_7seg(display_bcd(7 downto 4));
8     HEX2 <= bcd_to_7seg(display_bcd(11 downto 8));
9 end process;

```

O processo dos displays não precisa ser clockado, pois seu papel é apenas refletir o estado atual das chaves. Já o envio UART precisa capturar uma fotografia do valor em um instante específico, para evitar que uma mesma linha misture dígitos de leituras diferentes.

Convenção dos displays

Neste exemplo, os displays são ativos em nível baixo. Por isso, o padrão para o dígito 0 é "1000000": o segmento apagado fica em nível alto, e os segmentos acesos ficam em nível baixo.

1.5 Transmissor UART 8N1

UART é uma forma de comunicação serial assíncrona: transmissor e receptor não compartilham um sinal de clock, mas concordam previamente com a taxa de transmissão e com o formato do quadro. A configuração 8N1 usa 8 bits de dados, nenhuma paridade e 1 bit de parada. Nessa configuração, cada byte útil ocupa 10 tempos de bit: 1 bit de início, 8 bits de dados e 1 bit de parada [2].

Quando a linha está ociosa, ela permanece em nível alto. Para iniciar um quadro, o transmissor coloca a linha em nível baixo durante um tempo de bit. Em seguida, transmite os 8 bits de dados, começando pelo bit menos significativo. Por fim, coloca a linha em nível alto durante o bit de parada [2, 1]. A Figura 1.3 representa essa sequência temporal: primeiro vem o bit de início, depois os bits de dados b0 a b7, e finalmente o bit de parada.

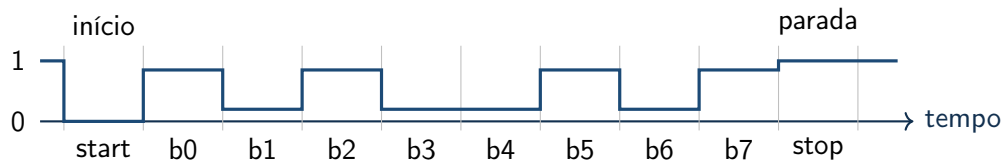


Figura 1.3: Estrutura conceitual de um quadro UART 8N1.

O transmissor do exemplo monta esse quadro em `tx_reg`. A concatenação `'1' & tx_data & '0'` coloca o bit de parada na posição mais alta, os 8 bits de dados no meio e o bit de início na posição mais baixa. Como o processo envia `tx_reg(bit_count)` começando em `bit_count = 0`, o bit de início sai primeiro.

Código 1.4: Núcleo do transmissor UART.

```

1 if tx_load = '1' and busy = '0' then
2     tx_reg    <= '1' & tx_data & '0';
3     busy      := '1';
4     bit_count := 0;
5 elsif tx_en = '1' and busy = '1' then
6     if bit_count < 10 then
7         tx      <= tx_reg(bit_count);
8         bit_count := bit_count + 1;
9     else
10        tx      <= '1';
11        tx_done  <= '1';
12        bit_count := 0;
13        busy     := '0';
14    end if;
15 end if;

```

A variável `busy` é interna ao processo. Ela impede que um novo byte seja carregado enquanto o quadro atual ainda está sendo transmitido. O sinal `tx_done`, por outro lado, é visível para a máquina de estados de envio: ele gera um pulso de um ciclo quando o transmissor terminou o byte atual.

1.6 Geração do baud rate

O transmissor não deve mudar de bit a cada ciclo de clock de 50 MHz. Ele só deve avançar no ritmo definido por `baudrate`. Por isso, há um processo separado que conta ciclos de clock e produz um pulso `tx_en` de um ciclo quando chega o instante de transmitir o próximo bit.

O número de ciclos de clock por bit é calculado por:

$$\text{baud_cycles} = \frac{\text{frequency}}{\text{baudrate}}.$$

Com os valores padrão, $50\,000\,000/115\,200 \approx 434$. O uso de divisão inteira produz uma aproximação. Para atividades didáticas e baud rates comuns, esse erro costuma ser pequeno o bastante para a comunicação funcionar com folga.

Código 1.5: Gerador de pulso na frequência de baud.

```

1 process(clk, rst_n)
2     variable baud_count : integer range 0 to baud_cycles := 0;
3 begin
4     if rst_n = '0' then
5         tx_en      <= '0';
6         baud_count := 0;
7     elsif rising_edge(clk) then
8         if baud_count < baud_cycles - 1 then
9             baud_count := baud_count + 1;
10            tx_en      <= '0';
11        else
12            baud_count := 0;
13            tx_en      <= '1';
14        end if;
15    end if;
16 end process;
```

Separação de responsabilidades

O gerador de baud não sabe qual byte está sendo enviado. O transmissor UART não sabe quando uma nova linha deve começar. A FSM de envio não sabe como serializar cada bit. Essa separação deixa cada processo menor e mais fácil de depurar.

1.7 Máquina de estados de envio

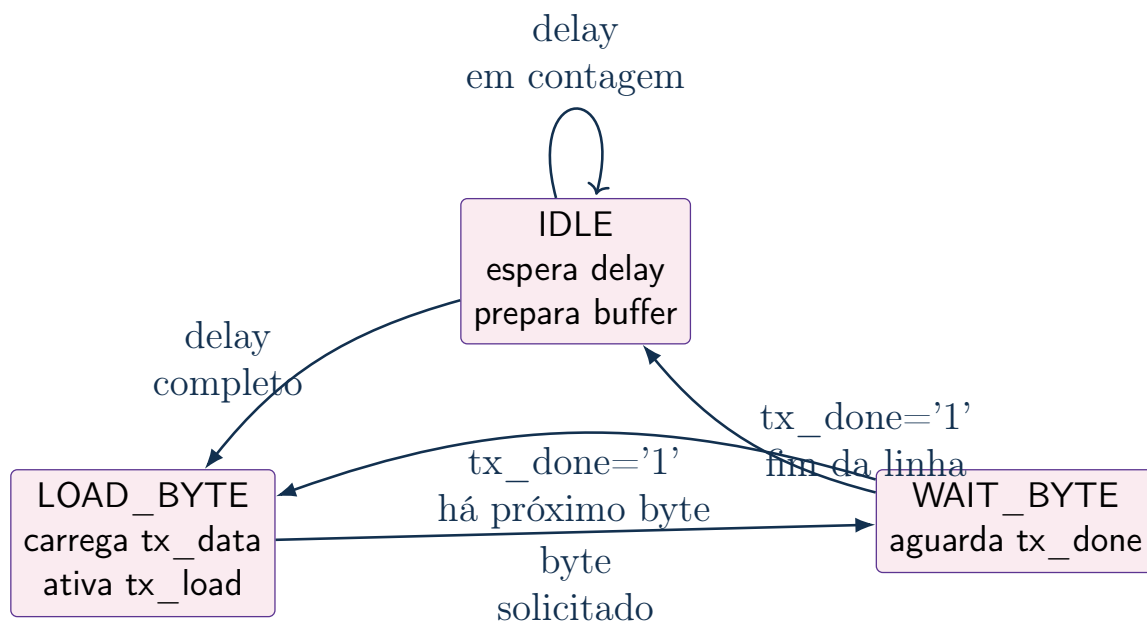
A transmissão de uma linha completa é controlada por uma máquina de estados finitos. Em termos conceituais, uma FSM descreve um circuito cujo comportamento depende do estado atual e das entradas observadas naquele instante. A organização usada aqui se aproxima de uma máquina de Moore, pois as ações principais estão associadas aos estados do controle, embora algumas decisões de transição dependam de sinais internos como `tx_done` [5, 4].

O tipo enumerado torna os estados legíveis no código:

Código 1.6: *Estados usados para controlar o envio da linha.*

```
1 type state_type is (IDLE, LOAD_BYTE, WAIT_BYTE);
```

O estado IDLE espera o tempo definido por `delay_ms`. Quando esse tempo termina, o circuito captura o valor das chaves, converte para BCD, monta um pequeno buffer de caracteres ASCII e escolhe de qual posição o envio deve começar. O estado `LOAD_BYTE` entrega um byte ao transmissor UART. O estado `WAIT_BYTE` aguarda o pulso `tx_done`; quando o byte acaba, a FSM decide se volta para IDLE ou se carrega o próximo caractere. A Figura 1.4 organiza essas transições e deixa claro que a permanência em `WAIT_BYTE` depende do término do byte atual.

**Figura 1.4:** *Transições principais da FSM que controla o envio dos bytes.*

Uma decisão importante aparece no fim do estado IDLE: o circuito remove zeros à esquerda. O buffer sempre é montado com centena, dezena e unidade, mas `char_index` escolhe onde a transmissão começa. Assim, 7 é enviado como "7", e não como "007"; 42 é enviado como "42", e não como "042".

Código 1.7: *Escolha do primeiro dígito transmitido.*

```

1 if captured_bcd(11 downto 8) /= "0000" then
2     char_index <= 0; -- envia centena, dezena e unidade
3 elsif captured_bcd(7 downto 4) /= "0000" then
4     char_index <= 1; -- envia dezena e unidade
5 else
6     char_index <= 2; -- envia apenas unidade
7 end if;
```

O caso do valor zero merece atenção: como centena e dezena são zero, o circuito começa em `char_index = 2`. Isso preserva um caractere útil na transmissão. Portanto, o valor 0 é enviado como "0", e não como uma linha vazia.

1.8 Buffer ASCII e fim de linha

A UART transmite bytes. Para enviar o número decimal como texto, cada dígito BCD precisa virar um caractere ASCII. Os caracteres '0' até '9' têm códigos de `x"30"` até `x"39"`. Como cada dígito BCD ocupa 4 bits, basta concatenar `x"3"` com o dígito.

Código 1.8: *Montagem dos dígitos decimais como caracteres ASCII.*

```
1 tx_buffer(0) <= x"3" & captured_bcd(11 downto 8);
2 tx_buffer(1) <= x"3" & captured_bcd(7 downto 4);
3 tx_buffer(2) <= x"3" & captured_bcd(3 downto 0);
```

Depois dos dígitos, o exemplo pode enviar um terminador de linha. Essa opção é controlada por `SW(9 downto 8)`, usando as duas chaves que não fazem parte do valor numérico. A Tabela 1.1 lista as quatro combinações possíveis e o efeito de cada uma no fim da mensagem transmitida.

SW(9 downto 8)	Bytes enviados após o número	Uso típico
"00"	LF x"0A"	nova linha em muitos terminais
"01"	CR x"0D"	retorno de carro
"10"	CRLF x"0D"x"0A"	convenção comum em terminais seriais
"11"	nenhum	envio apenas dos dígitos

Tabela 1.1: Seleção do terminador de linha.

A escolha do terminador também define `last_index`, isto é, a última posição do buffer que deve ser transmitida. Quando a opção é CRLF, o buffer usa cinco posições: três dígitos, CR e LF. Quando não há terminador, a última posição é o dígito da unidade.

Código 1.9: *Escolha do terminador e da última posição do buffer.*

```
1 case SW(9 downto 8) is
2   when "00" =>
3     tx_buffer(3) <= x"0A"; -- LF
4     last_index  <= 3;
5
6   when "01" =>
7     tx_buffer(3) <= x"0D"; -- CR
8     last_index  <= 3;
9
10  when "10" =>
11    tx_buffer(3) <= x"0D"; -- CR
12    tx_buffer(4) <= x"0A"; -- LF
13    last_index  <= 4;
14
15  when others =>
16    last_index  <= 2;      -- sem fim de linha
17 end case;
```

1.9 Testbench do exemplo

O arquivo `uart_sw_tb.vhd` testa o circuito sem depender da placa. Em um testbench, o circuito testado costuma ser chamado de DUT, de *Device Under Test*, ou dispositivo sob teste. Para deixar a simulação rápida, o testbench troca os valores reais dos *generics* por valores menores: clock de 1 kHz, baud rate de 100 baud e atraso entre envios de 1 ms. O comportamento lógico continua o mesmo, mas a simulação termina em muito menos tempo.

Código 1.10: *Instanciação do DUT com parâmetros reduzidos para simulação.*

```
1 dut : entity work. uart_sw
2     generic map (
3         frequency => frequency_tb,
4         baudrate  => baudrate_tb,
5         delay_ms  => delay_ms_tb
6     )
7     port map (
8         MAX10_CLK1_50 => clk,
9         KEY            => key,
10        SW             => sw,
11        LEDR           => ledr,
12        ARDUINO_IO     => arduino_io,
13        HEX0           => hex0,
14        HEX1           => hex1,
15        HEX2           => hex2
16    );
```

A parte mais interessante do testbench é a leitura do sinal serial. O procedimento `recebe_byte` espera uma borda de descida em `rx`, que corresponde ao bit de início. Depois ele avança para o meio do primeiro bit de dados e amostra os 8 bits seguintes. Essa estratégia imita o comportamento básico de um receptor UART em simulação.

Código 1.11: *Procedimento que recebe um byte UART no testbench.*

```
1 procedure recebe_byte(
2     signal rx : in std_logic;
3     variable ch : out std_logic_vector(7 downto 0)
4 ) is
5 begin
6     wait until falling_edge(rx);
7
8     wait for bit_period + bit_period / 2;
9     for i in 0 to 7 loop
10         ch(i) := rx;
11         wait for bit_period;
12     end loop;
13
14     assert rx = '1'
15         report "Bit de parada da UART deveria estar em nível alto"
16         severity error;
17 end procedure;
```

Embora o documento da disciplina possa apresentar `assert` em outro momento, aqui vale observar a utilidade prática: ele transforma uma verificação em uma mensagem automática de erro. A lógica de teste fica concentrada no procedimento `confere_byte`, que recebe um byte da UART e compara com o valor esperado.

Código 1.12: *Exemplos de estímulos usados no testbench.*

```

1 sw <= std_logic_vector(to_unsigned(42, sw'length));
2 confere_byte(arduino_io(1), x"34", "Falha no valor 42: esperado caractere
  '4'");
3 confere_byte(arduino_io(1), x"32", "Falha no valor 42: esperado caractere
  '2'");
4 confere_byte(arduino_io(1), x"0A", "Falha no valor 42: esperado LF");
5
6 sw <= "10" & std_logic_vector(to_unsigned(5, 8));
7 confere_byte(arduino_io(1), x"35", "Falha no valor 5 com CRLF: esperado
  caractere '5'");
8 confere_byte(arduino_io(1), x"0D", "Falha no valor 5 com CRLF: esperado CR");
9 confere_byte(arduino_io(1), x"0A", "Falha no valor 5 com CRLF: esperado LF");

```

O primeiro bloco confere se o valor 42 é transmitido como os caracteres '4', '2' e LF. O segundo bloco usa `SW(9 downto 8) = "10"` e confirma o envio de CRLF. Esses testes cobrem tanto a remoção de zeros à esquerda quanto a seleção do terminador de linha.

1.10 Relação entre processos e sinais

O exemplo é formado por processos concorrentes. Cada processo tem uma responsabilidade local, mas eles se comunicam por sinais. A Figura 1.5 resume essa relação. As entradas vêm da placa, os processos implementam o comportamento, os sinais internos sincronizam os blocos e as saídas vão para os displays, LEDs e pino serial.

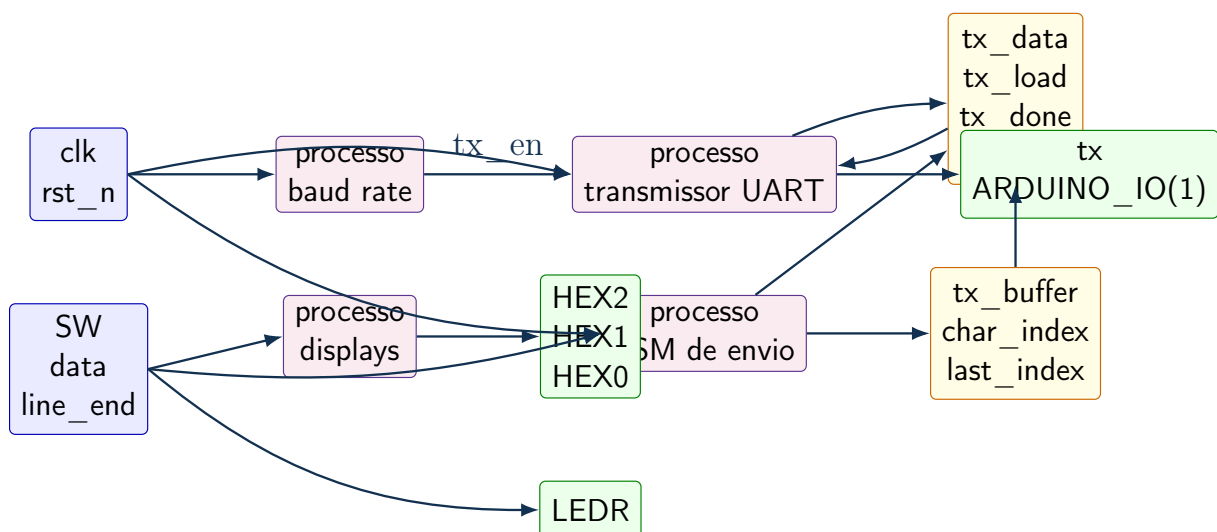


Figura 1.5: *Conexões principais entre processos, sinais internos, entradas e saídas.*

Sinais e variáveis

`tx_done` é sinal porque precisa sair do processo transmissor e ser lido pela FSM. Já `busy` pode ser variável, pois só interessa dentro do processo transmissor. Essa diferença é uma boa oportunidade para discutir escopo: sinais conectam blocos concorrentes; variáveis organizam cálculo local dentro de um processo.

1.11 Pontos de atenção para estudo

Este exemplo reúne vários conceitos que aparecem separadamente ao longo da disciplina:

- **Conversão numérica:** `to_bcd` transforma o valor binário das chaves em três dígitos BCD.
- **Reuso de função:** a mesma função de conversão é usada pelos displays e pela FSM de envio.
- **Codificação para display:** `bcd_to_7seg` converte cada dígito para segmentos ativos em nível baixo.
- **Comunicação serial:** o transmissor monta quadros UART 8N1, enviando primeiro o bit menos significativo.
- **Controle sequencial:** a FSM decide quando montar a linha, quando carregar um byte e quando voltar ao repouso.
- **Sincronização entre blocos:** `tx_load`, `tx_done` e `tx_en` conectam processos com responsabilidades diferentes.

Uma leitura proveitosa do código é seguir um valor concreto, por exemplo 42. Nos displays, o valor passa por `to_bcd` e vira 0, 4, 2. Na UART, a FSM captura o mesmo BCD, monta os caracteres `x"30"`, `x"34"` e `x"32"`, descarta o zero à esquerda iniciando em `char_index = 1`, acrescenta o terminador selecionado e entrega cada byte ao transmissor.

Do ponto de vista didático, o exemplo também mostra uma escolha de arquitetura: manter a conversão para os displays combinacional e registrar apenas a sequência que será enviada pela UART. Isso evita misturar duas leituras diferentes dentro da mesma linha serial, mas mantém a visualização local responsiva às chaves.

Referências Bibliográficas

- [1] Geoffrey Hunter. Uart communication protocol. <https://blog.mbedded.ninja/electronics/communication-protocols/uart-communication-protocol/>, 2011. Acesso em 28 maio 2026.
- [2] KU Leuven COSIC. Frame format. https://labsite-af5ab6.pages.gitlab.kuleuven.be/Lab2/lab2_4/, 2024. Acesso em 28 maio 2026.
- [3] Wikipedia contributors. Double dabble. https://en.wikipedia.org/wiki/Double_dabble, 2026. Acesso em 28 maio 2026.
- [4] Wikipedia contributors. Mealy machine. https://en.wikipedia.org/wiki/Mealy_machine, 2026. Acesso em 28 maio 2026.
- [5] Wikipedia contributors. Moore machine. https://en.wikipedia.org/wiki/Moore_machine, 2026. Acesso em 28 maio 2026.